

โครงงาน

Ice Cream Shop Game

จัดทำโดย

นางสาวกันตาพร จันทราภิรมย์ รหัสนักศึกษา 6804062610070

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

สารบัญ

ข้อมูลโครงงาน 1

บทที่ 1 บทนำ 1

ที่มาและความสำคัญของโครงการ 1

ประเภทของโครงการ 1

ประโยชน์ 1

ขอบเขตของโครงการ (Proposal) 1

บทที่ 2 การพัฒนา 3

2.1 เนื้อเรื่องย่อและวิธีการเล่น 3

2.2 คลาสไดอะแกรมและคำอธิบาย 4

2.3 รูปแบบการพัฒนา 5

2.4 อธิบายส่วนของโปรแกรม (แนวคิด OOP) 6

2.5 หน้าจอ GUI และ Component 9

2.6 Event Handling 11

2.7 อัลกอริทึมที่สำคัญในโปรแกรม 11

บทที่ 3 สรุป 14

ปัญหาที่พบระหว่างการพัฒนา 14

จุดเด่นของโปรแกรม 14

คำแนะนำสำหรับอาจารย์ผู้สอน 14

ข้อมูลโครงงาน

ชื่อโปรเจค: Ice Cream Shop Game

จัดทำโดย: นางสาวกันตาพร จันทราภิรมย์

อาจารย์ผู้สอน: ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

Source Code: https://github.com/Kantaporn-Jantrapirom/Ice-Cream-Shop-Game.git

บทที่ 1 บทนำ

ที่มาและความสำคัญของโครงการ

โครงงานนี้จัดทำขึ้นเพื่อวัดผลความสามารถในการเรียนวิชา Object-Oriented Programming โดยนำเนื้อหาที่เรียนมาสร้างเป็นเกม ใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุ และทำให้ผู้จัดทำได้เรียนรู้เครื่องมือต่างๆ ที่ใช้พัฒนาเกม ได้แก่ Java Swing, Graphics2D และระบบเสียง

ประเภทของโครงการ

โครงงานพัฒนาเกม (Game Development Project)

ประโยชน์

- ช่วยฝึกสมาธิและไหวพริบ
- ช่วยฝึกความจำ
- ให้ความบันเทิง
- ช่วยฝึกการบริหารจัดการสิ่งต่างๆ

ขอบเขตของโครงการ (Proposal)

ความต้องการของระบบ (Functional Requirements)

- กดเลือกวัตถุดิบและตัวลูกค้าได้
- แสดงผลลัพธ์หลังการกดเลือก และคำนวณราคาได้ถูกต้อง

- จับเวลาการเล่นต่อด่าน เวลาความอดทนของลูกค้า และเวลาที่ลูกค้าแต่ละคนเข้าร้าน
- นับจำนวนลูกค้าที่เสิร์ฟได้ในแต่ละด่าน
- มีเสียงเพลงที่เปลี่ยนไปตามด่านและเสียงเอฟเฟ็กต์

ตารางเวลาแผนการดำเนินโครงการ (Project Schedule)

ลำดับ	รายการ	15 - 18	20 - 5	12 - 25
1	หารูปตัวละครและทำกราฟิกต่างๆ			
2	ศึกษาเอกสารและข้อมูลที่เกี่ยวข้อง			
3	ลงมือเขียนโปรแกรม			
4	จัดทำเอกสาร			
5	ตรวจสอบและแก้ไขข้อผิดพลาด			

บทที่ 2 การพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

เนื้อเรื่องย่อ

ผู้เล่นรับบทเป็นเจ้าของร้านไอศกรีมที่เพิ่งเปิดใหม่ ลูกค้าจะเดินเข้ามาที่หน้าร้านทีละคน และแต่ละคนอยากได้ไอศกรีมไม่เหมือนกัน

ผู้เล่นต้องทำให้ตรงกับออเดอร์อย่างรวดเร็วและแม่นยำก่อนที่ลูกค้าจะหมดความอดทนแล้วเดินจากไป

เป้าหมายคือเสิร์ฟลูกค้าให้ครบ 15 คนภายใน 5 นาทีเพื่อผ่านด่าน เกมมี 3 ด่าน

และแต่ละด่านจะมีวัตถุดิบเพิ่มขึ้นเรื่อยๆ

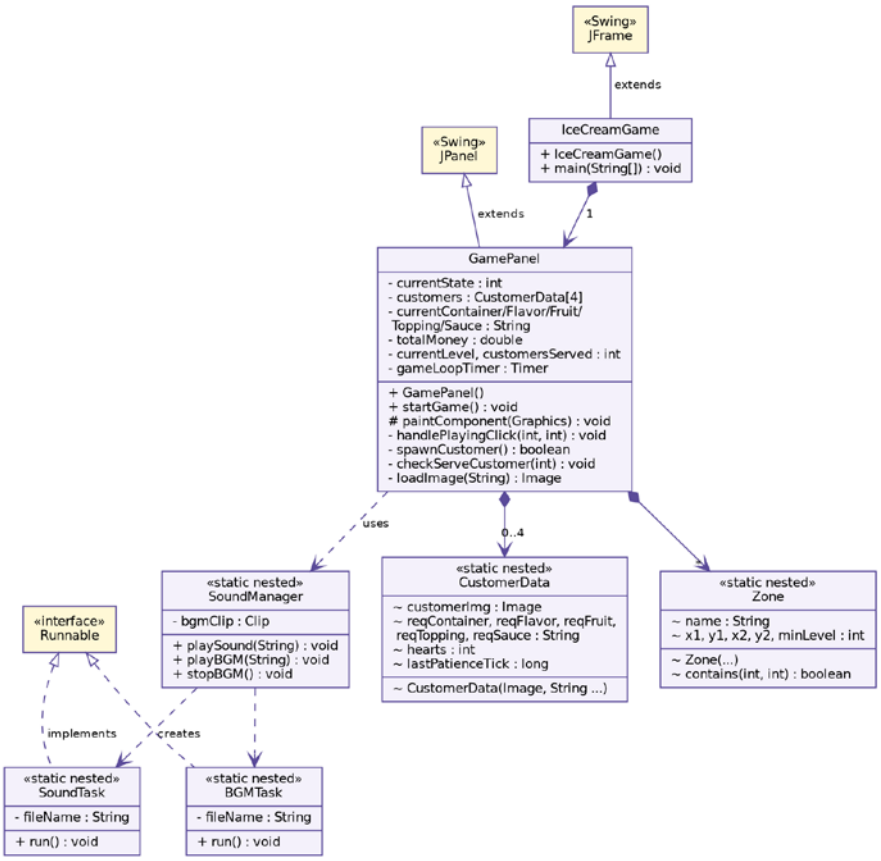
วิธีการเล่น

1. คลิกที่หน้าจอเริ่มต้นเพื่อเริ่มเกมที่ด่าน 1
2. ลูกค้าเข้าร้านทีละคน อยู่ในร้านพร้อมกันได้ไม่เกิน 4 คน แต่ละคนมีบับเบิลแสดงออเดอร์ (ภาชนะ รสไอศกรีม ผลไม้ ท็อปปิ้ง ซอส) และมีหัวใจ 5 ดวงแสดงความอดทน
3. คลิกเลือกวัตถุดิบจากเคาน์เตอร์ โดยต้องเลือกภาชนะ (Cone หรือ Cup) ก่อน ตามด้วยรสไอศกรีม แล้วจึงเลือกผลไม้ ท็อปปิ้ง และซอสตามที่ลูกค้าสั่ง (ถ้าลูกค้าไม่สั่งก็ไม่ต้องเลือก) เลือกได้ประเภทละ 1 อย่าง และเปลี่ยนไม่ได้ ต้องกดปุ่มล้างถาดที่มุมซ้ายล่างเพื่อเริ่มใหม่
4. เมื่อทำเสร็จ ให้คลิกที่ตัวลูกค้าที่ต้องการเสิร์ฟ ถ้าออเดอร์ถูกต้องจะได้เงิน ราคาพื้นฐาน 25 บาท ถ้าลูกค้ามีหัวใจ 4 ดวงขึ้นไปได้ทิปสุ่มเพิ่ม 10–30 บาท หัวใจ 2–3 ดวงได้ราคาเต็ม และหัวใจ 1 ดวงได้ครึ่งราคา
5. ถ้าเสิร์ฟผิด ลูกค้าจะเดินออกจากร้านโดยไม่ได้เงิน และถาดจะถูกล้างให้ใหม่
6. หัวใจของลูกค้าลดลง 1 ดวงทุก 6 วินาที ถ้าหัวใจหมดลูกค้าจะเดินออกจากร้านทันที
7. ชนะด่านเมื่อเสิร์ฟครบ 15 คนภายใน 5 นาที แล้วไปด่านถัดไป ถ้าหมดเวลาโดยเสิร์ฟไม่ครบจะแพ้และเล่นด่านเดิมซ้ำ เมื่อผ่านครบ 3 ด่านสามารถเริ่มเล่นใหม่ตั้งแต่ด่าน 1 ได้

ความแตกต่างของแต่ละด่าน

รายการ	ด่าน 1	ด่าน 2	ด่าน 3
รสไอศกรีม	4 รส	5 รส (+Thaitea)	6 รส (+Mint)
ผลไม้	4 ชนิด	5 ชนิด (+Apple)	6 ชนิด (+Orange)
ท็อปปิ้ง	2 ชนิด	3 ชนิด (+Choco Chip)	4 ชนิด (+Jelly)
ซอส	2 ชนิด	3 ชนิด (+Blueberry)	4 ชนิด (+Caramel)
ช่วงเวลาที่ลูกค้าคนถัดไปเข้าร้าน	5–8 วินาที	4–6 วินาที	2–4 วินาที

2.2 คลาสไดอะแกรมและคำอธิบาย



รูปที่ 1 Class Diagram ของโปรแกรม

โปรแกรมมีคลาสที่เขียนขึ้นเองทั้งหมด 7 คลาส แบ่งเป็นคลาสระดับบนสุด 2 คลาส และ

คลาสซ้อน (static nested class) ภายใน GamePanel และ SoundManager อีก 5 คลาส

คลาส	ประเภท	หน้าที่
IceCreamGame	extends JFrame	หน้าต่างหลักขนาด 1024×768 และมีเมธอด main() เป็นจุดเริ่มโปรแกรม ภายในมี GamePanel เพียงตัวเดียว
GamePanel	extends JPanel	คลาสหลักของเกม คุมสถานะเกม (MENU, PLAYING, LEVEL_WIN, LEVEL_LOSE) ตรรกะการเล่น และวาดภาพทั้งหมดด้วย paintComponent()
SoundManager	static nested	จัดการเสียง มี playSound() เล่นเอฟเฟกต์ครั้งเดียว, playBGM() เล่นเพลงวนซ้ำ, stopBGM() หยุดเพลง
SoundTask / BGMTask	static nested, implements Runnable	งานเล่นเสียงเอฟเฟกต์ และงานเล่นเพลงพื้นหลัง ถูกส่งให้ Thread แยกทำงาน เพื่อไม่ให้หน้าจอค้าง
CustomerData	static nested	เก็บข้อมูลลูกค้า 1 คน คือภาพลูกค้า ออเดอร์ 5 อย่าง จำนวนหัวใจ และเวลาที่หัวใจลดล่าสุด
Zone	static nested	เก็บพื้นที่สี่เหลี่ยมที่คลิกได้ของวัตถุดิบแต่ละอย่าง พร้อมด่านต่ำสุดที่เริ่มใช้ได้ (minLevel) และเมธอด contains(x, y)

2.3 รูปแบบการพัฒนา

- **ภาษา:** Java
- **รูปแบบโปรแกรม:** Application (Desktop Application แบบ Standalone) ไม่ใช่ Applet
  เริ่มทำงานจาก main() ใน IceCreamGame แล้วสร้างหน้าต่าง JFrame โดยตรง และ Export เป็นไฟล์ .jar เพื่อรันได้เลย

- **Library:** Java Swing (javax.swing) และ AWT (java.awt) สำหรับหน้าต่างและวาดภาพ, javax.sound.sampled สำหรับเสียง
- **Game Loop:** ใช้ javax.swing.Timer ทำงานทุก 100 มิลลิวินาที เพื่ออัปเดตเกมแบบ real-time
- **Multithreading:** เล่นเสียงใน Thread แยก เพื่อไม่บล็อก UI thread
- **Assets:** ภาพโหลดผ่าน ImageIcon และ getResource() (รวมกว่า 50 ไฟล์) ส่วนเสียงเป็นไฟล์ .wav

2.4 อธิบายส่วนของโปรแกรม (แนวคิด OOP)

Constructor

ใช้กำหนดค่าเริ่มต้นให้อ็อบเจกต์ทันทีที่สร้าง โปรแกรมมี constructor ดังนี้

- IceCreamGame() ตั้งชื่อ ขนาด และตำแหน่งหน้าต่าง แล้วเพิ่ม GamePanel เข้าไป
- GamePanel() เป็น constructor ที่ทำงานมากที่สุด คือโหลดภาพกว่า 50 ไฟล์ สร้าง Timer ของเกม และผูก MouseListener
- CustomerData(...) รับภาพและออเดอร์ 5 อย่างมาเก็บ และบันทึกเวลาที่ลูกค้าเข้ามาไว้ใน lastPatienceTick ส่วนหัวใจเริ่มต้น 5 ดวงกำหนดไว้ที่ฟิลด์ hearts = 5
- Zone(...) มี 2 รูปแบบ (constructor overloading) แบบสั้นเรียก this(...) เพื่อกำหนด minLevel = 1 ให้อัตโนมัติ

```
CustomerData(Image img, String container, String flavor, String fruit, String topping, String sauce) {
    this.customerImg = img;
    this.reqContainer = container;   // ... flavor, fruit, topping, sauce
    this.lastPatienceTick = System.currentTimeMillis();
}
Zone(String name, int x1, int y1, int x2, int y2) {
    this(name, x1, y1, x2, y2, 1);   // minLevel เริ่มต้น = 1
}
```

Encapsulation

ฟิลด์ที่เก็บสถานะเกมของ GamePanel เป็น private ทั้งหมด เช่น currentState, customers, currentContainer, totalMoney ภายนอกจึงแก้ค่าโดยตรงไม่ได้ เมธอด public ของ GamePanel มีเพียง startGame() ส่วนการเสิร์ฟ คิดเงิน และสร้างลูกค้าทำผ่านเมธอด private ภายในคลาส (checkServeCustomer(), spawnCustomer()) นอกจากนี้ CustomerData, Zone, SoundTask และ BGMTask เป็น private static class ที่ใช้ได้เฉพาะภายใน GamePanel และตัวแปร bgmClip ใน SoundManager เป็น private เช่นกัน

```
public class GamePanel extends JPanel {
    private int currentState = STATE_MENU;
    private CustomerData[] customers = new CustomerData[4];
    private double totalMoney = 0.0;

    public void startGame() { ... }                    // public เมธอดเดียว
    private void checkServeCustomer(int clickX) { ... }  // ซ่อนในคลาส
}
```

Composition

โปรแกรมใช้ความสัมพันธ์แบบ "has-a" ได้แก่ IceCreamGame มี GamePanel อยู่ภายใน และ GamePanel มีอาเรย์ CustomerData สูงสุด 4 คน กับอาเรย์ Zone ของวัตถุดิบแต่ละด่าน โดย GamePanel เป็นผู้สร้างและลบอ็อบเจกต์ลูกค้าเอง (สร้างเมื่อลูกค้าเข้าร้าน และ null เมื่อลูกค้าออกจากร้านหรือได้รับการเสิร์ฟ) ลูกค้าจึงเป็นส่วนหนึ่งของ GamePanel ไม่ใช่การสืบทอด

```
add(new GamePanel());                                  // IceCreamGame has-a GamePanel
customers[targetSlot] = new CustomerData(img, c, f, fr, t, s);   // ลูกค้าเข้าร้าน
customers[index] = null;                               // ลูกค้าออกจากร้าน
```

Polymorphism

เกิดจากการ override เมธอดให้ทำงานต่างจากเดิมตามคลาสของตัวเอง เมื่อ Swing เรียก paintComponent() หรือ mouseClicked() ตอนรันไทม์ จะเรียกโค้ดที่เขียนทับไว้จริง (dynamic binding) และ new Thread(...) รับอ็อบเจกต์ชนิด Runnable ได้ทั้ง SoundTask และ BGMTask โดยแต่ละตัวมี run() ที่ทำงานต่างกัน นอกจากนี้ constructor ของ Zone ยังเป็น overloading อีกด้วย

```
@Override
protected void paintComponent(Graphics g) { ... }        // override JPanel

addMouseListener(new MouseAdapter() {
   @Override
   public void mouseClicked(MouseEvent e) { ... }        // override MouseAdapter
});

new Thread(new SoundTask(fileName)).start();             // Thread รับ Runnable
new Thread(new BGMTask(fileName)).start();               // คนละคลาส run() ต่างกัน
```

Abstract

ผู้จัดทำไม่ได้ประกาศ abstract class ขึ้นเอง แต่นำ abstract class และ interface ของ Java มาใช้งานจริง 4 จุด

1. MouseAdapter เป็น abstract class ที่เตรียมเมธอดว่างของ MouseListener ไว้ครบ โปรแกรมจึง override เฉพาะ mouseClicked() ที่ต้องใช้
2. Runnable เป็น interface ที่ SoundTask และ BGMTask เขียน run() ตามข้อกำหนด เพื่อส่งให้ Thread ทำงาน
3. ActionListener เป็น interface ที่ Timer รับไว้ โปรแกรมเขียนด้วย lambda (e -> { ... }) ระบุเฉพาะว่า "ทำอะไร" ทุก 100 มิลลิวินาที โดยไม่ต้องรู้ว่า Timer ทำงานเบื้องหลังอย่างไร

4. JFrame และ JPanel ที่โปรแกรมสืบทอดมา มีต้นทางจาก abstract class Component และ JComponent ของ AWT/Swing

```
private static class SoundTask implements Runnable {
    private String fileName;
    @Override
    public void run() { ... }
}
gameLoopTimer = new Timer(100, e -> { ... });   // ActionListener (interface) เขียนด้วย lambda
```

Inheritance

มีการสืบทอดจากคลาสของ Swing โดยตรง 2 จุด คือ IceCreamGame สืบทอดจาก JFrame และ GamePanel สืบทอดจาก JPanel รวมถึงการสร้าง anonymous class จาก MouseAdapter ส่วนการสืบทอดระหว่างคลาสที่เขียนขึ้นเองไม่มี

```
public class IceCreamGame extends JFrame { … }
public class GamePanel extends JPanel { … }
```

2.5 หน้าจอ GUI และ Component

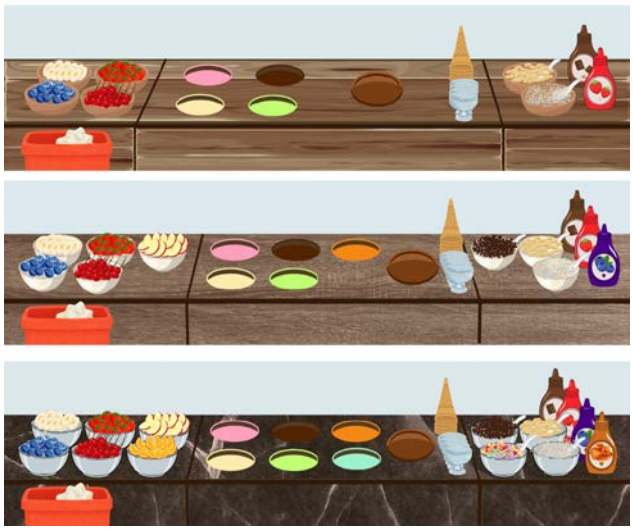
หน้าต่างหลักคือ JFrame (IceCreamGame) ขนาด 1024×768 พิกเซล ปรับขนาดไม่ได้ และแสดงกลางจอ ภายในมี GamePanel เพียงตัวเดียวที่เติมเต็มหน้าต่าง โปรแกรมไม่ได้ใช้ Swing component มาตรฐานอย่าง JButton, JLabel หรือ JTextField เลย แต่วาดทุกอย่างเองด้วย Graphics2D ใน paintComponent() แล้วตรวจจับการคลิกโดยเทียบพิกัด (x, y) กับกรอบสี่เหลี่ยมของวัตถุดิบแต่ละอย่าง (คลาส Zone) เรียกว่าเทคนิค custom painted GUI ซึ่งควบคุมหน้าตาได้อิสระ แต่ต้องกำหนดพิกัดเอง

หน้าจอที่แสดงถูกกำหนดจากตัวแปร currentState ของ GamePanel ดังนี้

สถานะ (state)	สิ่งที่วาดบนหน้าจอ
STATE_MENU	ภาพ 1.png เต็มจอ (คลิกที่ไหนก็เริ่มเกม) หากหาภาพไม่พบจะวาดพื้นสีฟ้าพร้อมข้อความแทน
STATE_PLAYING	พื้นหลังตามด่าน ⟶ ลูกค้า (สูงสุด 4 คน) พร้อมบับเบิลออเดอร์และหัวใจ ⟶ เคาน์เตอร์วัตถุดิบตามด่าน ⟶ แผงข้อมูล (เวลาที่เหลือ เงิน จำนวนลูกค้าที่เสิร์ฟ) ⟶ ถาดไอศกรีมที่กำลังทำ
STATE_LEVEL_WIN / LOSE	วาดฉากหลังสีดำโปร่งใสทับหน้าจอเกม แล้ววางป้ายผ่านด่านหรือป้ายแพ้ตรงกลาง ซึ่งป้ายนี้ทำหน้าที่เป็นปุ่มไปด่านถัดไปหรือเล่นซ้ำ



รูปที่ 2 หน้าจอเริ่มต้น (STATE_MENU)



รูปที่ 3 เคาน์เตอร์วัตถุดิบด่าน 1, 2 และ 3 (จากบนลงล่าง)



รูปที่ 4 บับเบิลออเดอร์และไอคอนหัวใจแสดงความอดทนของลูกค้า

2.6 Event Handling

Event	ใช้ทำอะไร
MouseAdapter.mouseClicked() (anonymous class)	จุดรับการโต้ตอบทั้งหมด เล่นเสียงคลิก แล้วแยกตาม state: MENU เริ่มเกม, PLAYING เรียก handlePlayingClick() เพื่อเลือกวัตถุดิบ ล้างถาด หรือเสิร์ฟ, LEVEL_WIN ไปด่านถัดไป, LEVEL_LOSE เล่นด่านเดิมซ้ำ
javax.swing.Timer ทุก 100 ms (ActionListener แบบ lambda)	game loop หลัก เช็คเวลาหมด สุ่มสร้างลูกค้าใหม่ ลดหัวใจลูกค้าที่รอนาน และสั่ง repaint()

2.7 อัลกอริทึมที่สำคัญในโปรแกรม

1. ตรวจสอบออเดอร์ (Order Matching)

เปรียบเทียบ 5 คุณสมบัติของไอศกรีมที่ทำกับที่ลูกค้าสั่ง แบบไม่สนตัวพิมพ์เล็กใหญ่ ต้องตรงทั้งหมดจึงถือว่าถูกต้อง

```
boolean isCorrect = currentContainer.equalsIgnoreCase(c.reqContainer) &&
                    currentFlavor.equalsIgnoreCase(c.reqFlavor) &&
                    currentFruit.equalsIgnoreCase(c.reqFruit) &&
                    currentTopping.equalsIgnoreCase(c.reqTopping) &&
                    currentSauce.equalsIgnoreCase(c.reqSauce);
```

2. คำนวณรายได้ตามความอดทนของลูกค้า (Tiered Pricing)

เงินที่ได้ขึ้นกับจำนวนหัวใจที่เหลือตอนเสิร์ฟ หัวใจ ≥ 4 ได้ 25 บาทบวกทิปสุ่ม 10–30 บาท, หัวใจ 2–3 ได้ราคาเต็ม, หัวใจ 1 ได้ครึ่งราคา

```
if (c.hearts >= 4) {
    int tip = 10 + random.nextInt(21);
    totalMoney += basePrice + tip;
} else if (c.hearts >= 2) {
    totalMoney += basePrice;
} else if (c.hearts == 1) {
    totalMoney += basePrice * 0.5;
}
```

3. ลดความอดทนตามเวลา (Patience Decay)

ทุก 6 วินาทีที่ลูกค้ารอ หัวใจจะลดลง 1 ดวง

ถ้าหัวใจหมดลูกค้าจะออกจากร้านทันทีและเปิดช่องให้คนใหม่ โดยตรวจใน game loop ทุก 100 มิลลิวินาที

```
if (c != null && now - c.lastPatienceTick > 6000) {
    c.hearts--;
    c.lastPatienceTick = now;
    if (c.hearts <= 0) {
        customers[i] = null;
        scheduleNextSpawn();
    }
}
```

4. ตรวจตำแหน่งคลิก (Coordinate Hit-Testing)

เก็บพื้นที่คลิกของวัตถุดิบเป็นอาเรย์ของ Zone แยกตามด่าน เมื่อคลิกจะวนหา Zone ที่มีพิกัดตรงกัน และข้าม Zone ที่ด่านปัจจุบันยังไม่ถึง minLevel (เช่น Mint ใช้ได้ตั้งแต่ด่าน 3)

ลำดับการเลือกถูกควบคุมด้วยการเช็ค isEmpty() คือต้องมีภาชนะ แล้วมีรสไอศกรีม จึงเลือกผลไม้ ท็อปปิ้ง ซอส หรือเสิร์ฟได้

```
private String pickZone(Zone[] zones, int x, int y) {
    for (Zone z : zones) {
        if (currentLevel >= z.minLevel && z.contains(x, y)) return z.name;
    }
    return null;
}
```

5. สุ่มสร้างลูกค้า (Random Spawn)

เมื่อถึงเวลา nextSpawnTime จะสุ่มเลือกช่องว่างจาก 4 ช่อง สุ่มภาชนะและวัตถุดิบจากรายการที่เปิดให้ใช้ในด่านนั้น (ผลไม้ ท็อปปิ้ง ซอส อาจไม่สั่งก็ได้) และสุ่มภาพลูกค้า จากนั้นสุ่มเวลาที่ลูกค้าคนถัดไปจะมาตามช่วงของแต่ละด่าน

6. เงื่อนไขชนะและแพ้ด่าน (Win/Lose Condition)

ชนะทันทีเมื่อเสิร์ฟครบ TARGET_CUSTOMERS (15 คน) ภายใน LEVEL_DURATION_MS (5 นาที) และเมื่อหมดเวลาจะตัดสินว่าชนะหรือแพ้จากจำนวนลูกค้าที่เสิร์ฟได้

```
currentState = (customersServed >= TARGET_CUSTOMERS) ? STATE_LEVEL_WIN :
STATE_LEVEL_LOSE;
```

บทที่ 3 สรุป

ปัญหาที่พบระหว่างการพัฒนา

- การกำหนดพิกัดคลิกของวัตถุดิบแต่ละอย่างทำได้ยาก
  เพราะภาพเคาน์เตอร์แต่ละด่านวางไม่เหมือนกัน และพิกัดบางจุดซ้อนทับกัน
  ทำให้กดวัตถุดิบหนึ่งแต่ขึ้นอีกอย่างหนึ่ง แก้โดยปรับพิกัดและแยกเป็นอาเรย์ Zone ตามด่าน

จุดเด่นของโปรแกรม

- **ความยากเพิ่มขึ้นตามด่าน** ทุกด่านเพิ่มรสชาติ ผลไม้ ท็อปปิ้ง และซอส พร้อมเปลี่ยนพื้นหลัง
  เคาน์เตอร์ ลูกค้า เพลง และเร่งเวลาที่ลูกค้าเข้าร้าน ความยากจึงเพิ่มขึ้นทีละขั้น ไม่ใช่แค่เร่งเวลา
- **วาดหน้าจอเองทั้งหมดด้วย Graphics2D** ไม่ใช้ JButton หรือ JLabel เลย
  ทำให้ได้หน้าตาเกมที่เป็นเอกลักษณ์
- **เก็บพิกัดและด่านของวัตถุดิบเป็นข้อมูลในคลาส Zone** แทนการเขียน if ซ้ำๆ
  ทำให้โค้ดเลือกวัตถุดิบสั้น และปรับพิกัดหรือด่านที่เริ่มใช้ (minLevel) ได้ง่าย
- **ระบบความอดทนของลูกค้าแบบเรียลไทม์** หัวใจลดทุก 6 วินาที
  ผู้เล่นต้องบริหารเวลากับลูกค้าสูงสุด 4 คนพร้อมกัน
- **ระบบเสียงแยกเป็นคลาสของตัวเอง** SoundManager เล่นเสียงใน Thread แยก
  ไม่บล็อกการวาดภาพและรับ input และสลับเพลงตามด่านได้ทันที

คำแนะนำสำหรับอาจารย์ผู้สอน

อยากให้อาจารย์สอนช้าลงอีกนิดค่ะ บางเรื่องเป็นเรื่องใหม่ จึงใช้เวลาทำความเข้าใจนานหน่อยค่ะ